

Lecture 8

Heapsort



Slides are based on the textbook and its notes

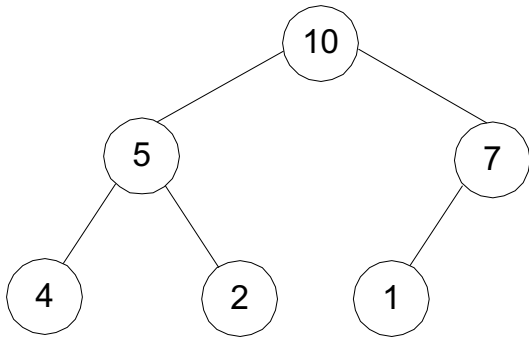
Overview

□ Heapsort

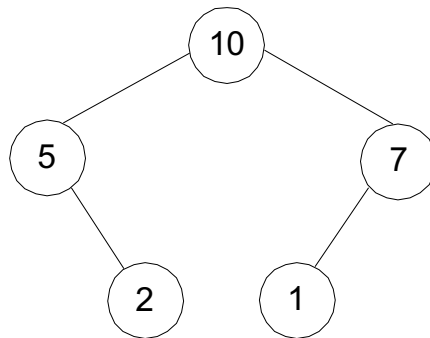
- $O(n \lg n)$ worst case – like mergesort.
 - Sorts in place – like insertion sort.
 - Combines the best of both algorithms.
-
- To understand heapsort, we will cover heaps and heap operations, and then we will take a look at priority queues.

Heaps

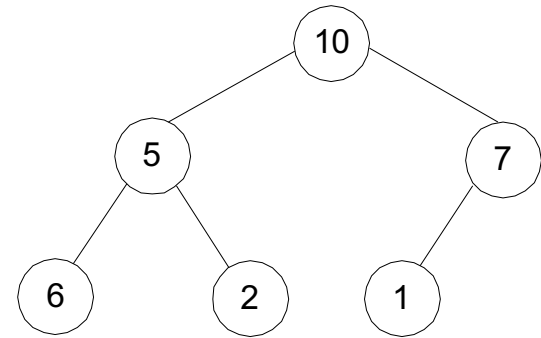
- A **(binary) heap** (not garbage-collected storage) is an array object that we can view as a nearly complete binary tree. Each node of the tree corresponds to an element of the array. The tree is completely filled on all levels except possibly the lowest, which is filled from the left up to a point. The key at each node is at least as large as ($\geq$) the keys at its children, such a heap is called a **max-heap**.



a max-heap



not a heap

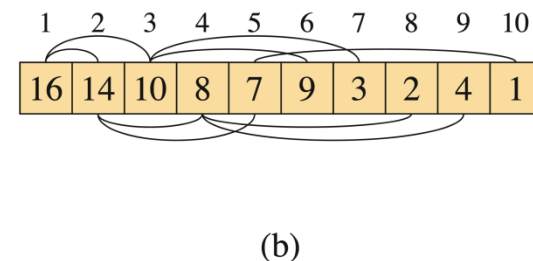
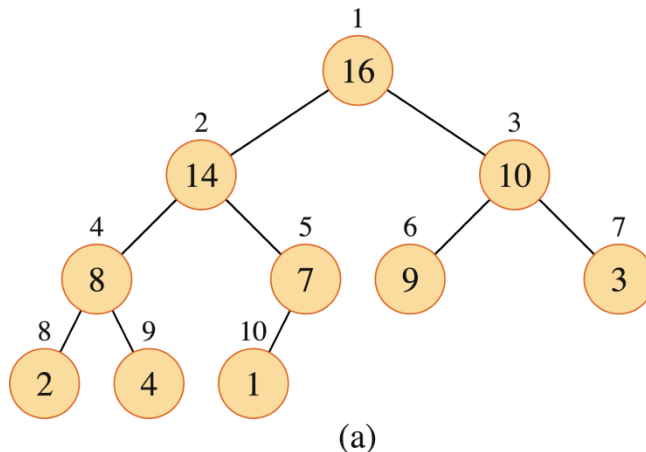


not a heap

- Heap's elements are ordered top down (along any path down from its root),₃ but they are not ordered left to right.

Heaps

- A heap can be stored as an array A .
 - **Height** of node = number of edges on a longest simple path from the node down to a leaf.
 - **Height** of heap = height of root = $\Theta(\lg n)$.
 - Root of tree is $A[1]$.
 - Parent of $A[i] = A[\lfloor i/2 \rfloor]$.
 - Left child of $A[i] = A[2i]$.
 - Right child of $A[i] = A[2i + 1]$.
 - Attribute: $A.heap\text{-}size$ says how many elements are stored in A . Only the elements in $A[1 : A.heap\text{-}size]$ are in the heap.



Heaps

- The root of the tree is $A[1]$, and given the index i of a node, there is simple way to compute the indices of its parent, left child, and right child with the one-line procedures PARENT, LEFT, and RIGHT.

PARENT(i)

1 **return** $\lfloor i/2 \rfloor$

LEFT(i)

1 **return** $2i$

RIGHT(i)

1 **return** $2i + 1$

- For max-heaps (largest element at root), *max-heap property*: for all nodes i , excluding the root, $A[\text{PARENT}(i)] \geq A[i]$.
- For min-heaps (smallest element at root), *min-heap property*: for all nodes i , excluding the root, $A[\text{PARENT}(i)] \leq A[i]$.

Maintaining the heap property

- MAX-HEAPIFY is important for manipulating max-heaps. It is used to maintain the max-heap property.
 - Before MAX-HEAPIFY, $A[i]$ may be smaller than its children.
 - Assume that left and right subtrees of i are max-heaps. (No violations of max-heap property within the left and right subtrees. The only violation within the subtree rooted at i could be between i and its children.)
 - After MAX-HEAPIFY, subtree rooted at i is a max-heap.

MAX-HEAPIFY(A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )
```

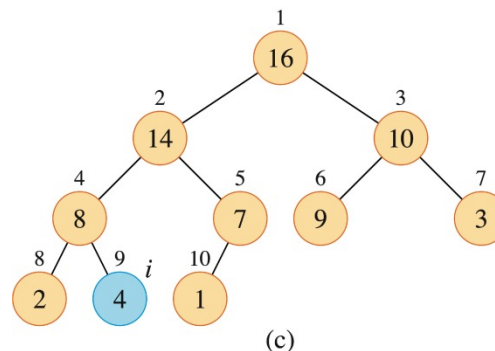
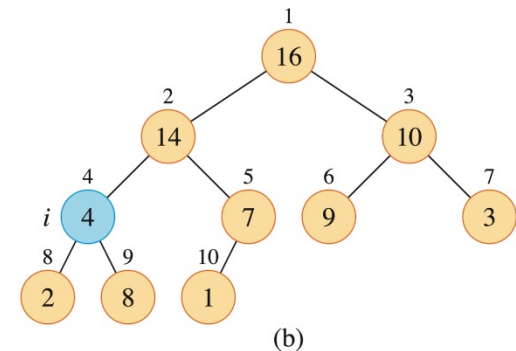
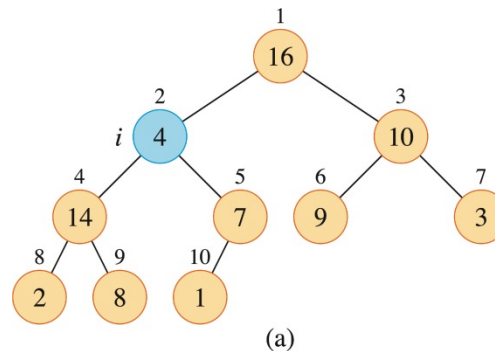
Maintaining the heap property

□ The way MAX-HEAPIFY works:

- Compare $A[i]$, $A[\text{LEFT}(i)]$, and $A[\text{RIGHT}(i)]$.
- If necessary, swap $A[i]$ with the larger of the two children to preserve heap property.
- Continue this process of comparing and swapping down the heap, until subtree rooted at i is max-heap. If we hit a leaf, then the subtree rooted at the leaf is trivially a max-heap.

□ *Example:*

- Node 2 violates the max-heap property.
- Compare node 2 with its children, and then swap it with the larger of the two children.
- Continue down the tree, swapping until the value is properly placed at the root of a subtree that is a max-heap. In this case the max-heap is a leaf.



Maintaining the heap property: Analysis

- Intuitively, heap is almost-complete binary tree, hence must process $O(\lg n)$ levels, with constant work at each level (comparing 3 items and maybe swapping 2).
- To formally analyze MAX-HEAPIFY, let $T(n)$ be the worst-case running time that the procedure takes on a subtree of size at most n .
- For a tree rooted at a given node i , the running time is the $\Theta(1)$ time to fix up the relationships among the elements $A[i]$, $A[\text{LEFT}(i)]$, and $A[\text{RIGHT}(i)]$, plus the time to run MAX-HEAPIFY on a subtree rooted at one of the children of node i (assuming that the recursive call occurs). Each of the children's subtrees has the size at most $2n/3$, and therefore we can describe the running time of MAX-HEAPIFY by the recurrence

$$T(n) \leq T(2n/3) + \Theta(1)$$

- The solution to this recurrence, by case 2 of the master theorem, is $T(n) = O(\lg n)$. Alternatively, we can characterize the running time of MAX-HEAPIFY on a node of height h as $O(h)$.

Building a heap

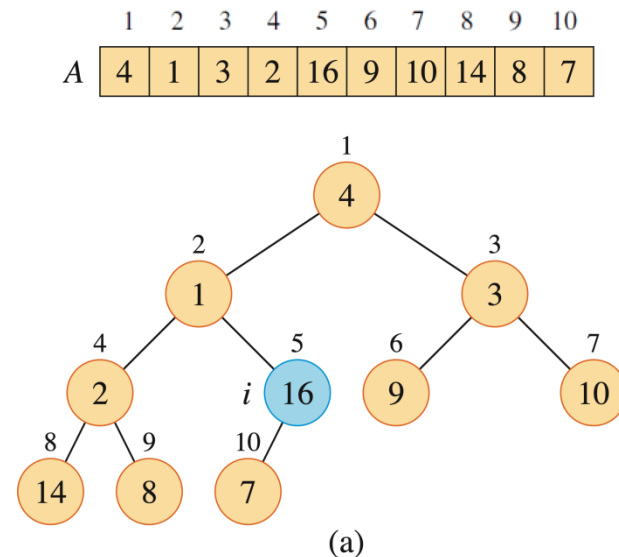
- The procedure BUILD-MAX-HEAP converts an array $A[1 : i]$ into a max-heap by calling MAX-HEAPIFY in a bottom-up manner.

BUILD-MAX-HEAP(A, n)

```
1   $A.heap-size = n$ 
2  for  $i = \lfloor n/2 \rfloor$  downto 1
3      MAX-HEAPIFY( $A, i$ )
```

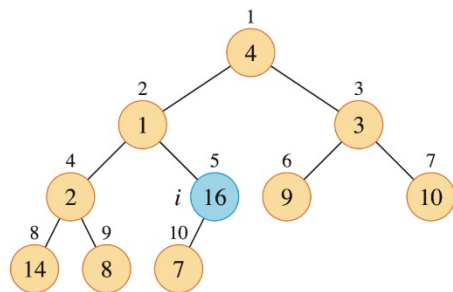
- **Example:** Building a max-heap by calling BUILD-MAX-HEAP($A, 10$).

- $A.heap-size$ is set to 10.
- i starts off as 5.
- MAX-HEAPIFY is applied to subtrees rooted at nodes (in order): $A[5]$, $A[4]$, $A[3]$, $A[2]$, and $A[1]$.

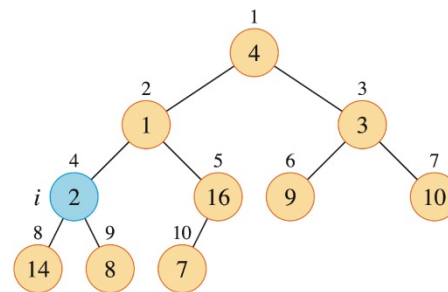


Building a heap: Example

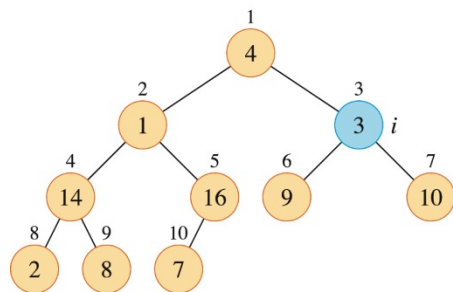
	1	2	3	4	5	6	7	8	9	10
A	4	1	3	2	16	9	10	14	8	7



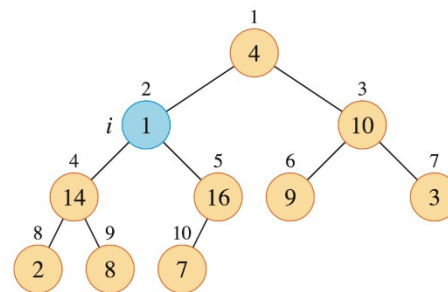
(a)



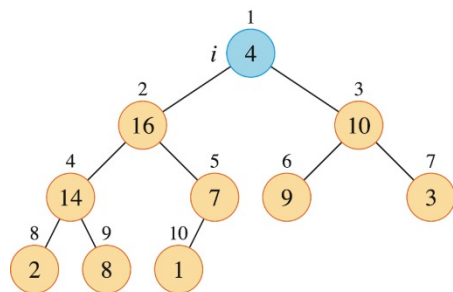
(b)



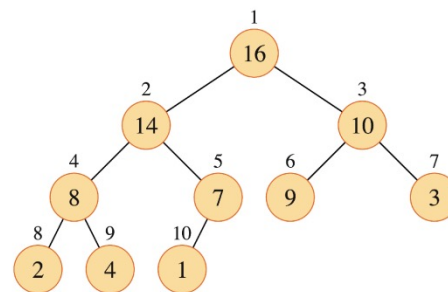
(c)



(d)



(e)



(f)

Building a heap: Correctness

- ❑ **Loop invariant:** At start of each iteration of the **for** loop of lines 2-3, each node $i + 1, i + 2, \dots, n$ is the root of a max-heap.
- ❑ **Initialization:** Prior to the first iteration of the loop, $i = \lfloor n/2 \rfloor$. Each node $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2, \dots, n$ is a leaf, which is the root of a trivial max-heap. Thus the invariant is initially true.
- ❑ **Maintenance:** The children of node i are indexed higher than i , so by the loop invariant, they are both roots of max-heaps. This is precisely the condition required for the call $\text{MAX-HEAPIFY}(A, i)$ to make node i a max-heap root. Moreover, the MAX-HEAPIFY call preserves the property that nodes $i + 1, i + 2, \dots, n$ are all the root of a max-heap. Decrementing i in the **for** loop update reestablishes the loop invariant for the next iteration.
- ❑ **Termination:** The loop makes exactly $\lfloor n/2 \rfloor$ iterations, and so it terminates. At termination, $i = 0$. By the loop invariant, each node $1, 2, \dots, n$ is the root of a max-heap. In particular, node 1 is.

Building a heap: Analysis

- Simple bound:** $O(n)$ calls to MAX-HEAPIFY, each of which takes $O(\lg n)$ time $\Rightarrow O(n \lg n)$.
 (A good approach to analysis in general is to start by proving an easy bound, then try to tighten it).
- Tighter analysis:** Observation: Time to run MAX-HEAPIFY at a node varies with the height of the node in the tree, and that the heights of most nodes are small. As an n -element heap has height $\lfloor \lg n \rfloor$ and at most $\lceil n/2^{h+1} \rceil$ nodes of any height h .
- The time required by MAX-HEAPIFY when called on a node of height h is $O(h)$.
- Letting c be the constant implicit in the asymptotic notation, we can express the total cost of BUILD-MAX-HEAP as being bound from above by $\sum_{h=0}^{\lfloor \lg n \rfloor} \lceil n/2^{h+1} \rceil ch$.
- As $\lceil n/2^{h+1} \rceil \geq 1/2$ for $0 \leq h \leq \lfloor \lg n \rfloor$ and since $\lceil x \rceil \leq 2x$ for any $x \geq 1/2$, we have $\lceil n/2^{h+1} \rceil \leq n/2^h$.
 We thus obtain

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil ch$$

$$\leq \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{n}{2^h} ch$$

$$= cn \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h}$$

$$\leq cn \sum_{h=0}^{\infty} \frac{h}{2^h}$$

$$\leq cn \cdot \frac{1/2}{(1-1/2)^2} \quad (\text{by equation (A.11) on page 1142 with } x = 1/2)$$

$$= O(n).$$

- Hence, we can build a max-heap from an unordered array in linear time.

$$\sum_{k=0}^{\infty} kx^k = \frac{x}{(1-x)^2}$$

for $|x| < 1$.

The Heapsort algorithm

- Given an input array, the heapsort algorithm acts as follows:
 - Builds a max-heap from the array.
 - Starting with the root (the maximum element), the algorithm places the maximum element into the correct place in the array by swapping it with the element in the last position in the array.
 - “Discard” this last node (knowing that it is in its correct place) by decreasing the heap size, and calling MAX-HEAPIFY on the new (possibly incorrectly-placed) root.
 - Repeat this “discarding” process until only one node (the smallest element) remains, and therefore is in the correct place in the array.

HEAPSORT(A, n)

```
1  BUILD-MAX-HEAP( $A, n$ )
2  for  $i = n$  downto 2
3      exchange  $A[1]$  with  $A[i]$ 
4       $A.heap-size = A.heap-size - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
```

The Heapsort algorithm: Example 1



HEAPSORT(A, n)

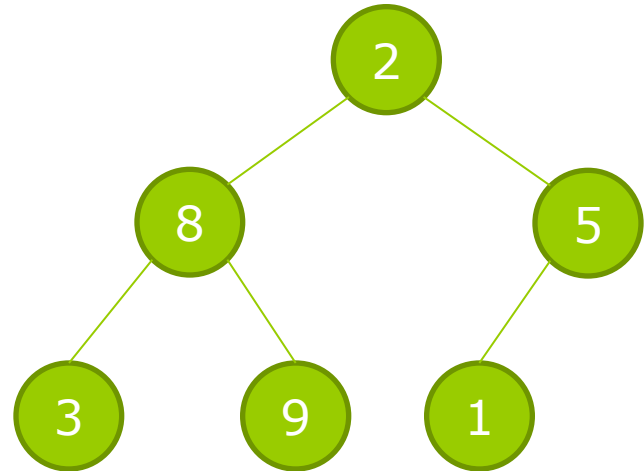
```
1  BUILD-MAX-HEAP( $A, n$ )
2  for  $i = n$  downto 2
3      exchange  $A[1]$  with  $A[i]$ 
4       $A.heap-size = A.heap-size - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
```

The Heapsort algorithm: Example 1



HEAPSORT(A, n)

```
1  BUILD-MAX-HEAP( $A, n$ )
2  for  $i = n$  downto 2
3      exchange  $A[1]$  with  $A[i]$ 
4       $A.heap\text{-}size = A.heap\text{-}size - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
```

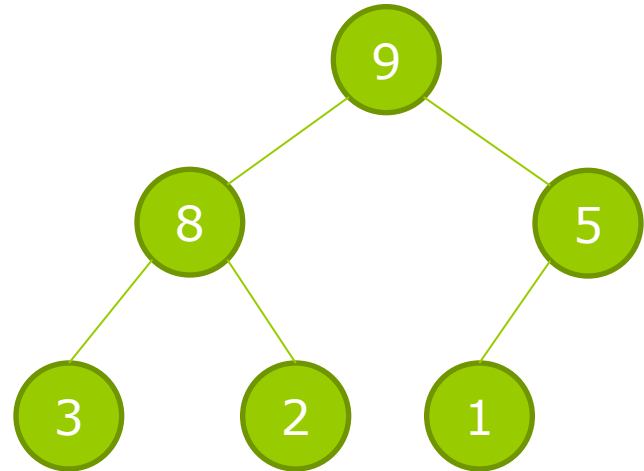


The Heapsort algorithm: Example 1



HEAPSORT(A, n)

```
1  BUILD-MAX-HEAP( $A, n$ )
2  for  $i = n$  downto 2
3      exchange  $A[1]$  with  $A[i]$ 
4       $A.heap\text{-}size = A.heap\text{-}size - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
```



The Heapsort algorithm: Example 1



HEAPSORT(A, n)

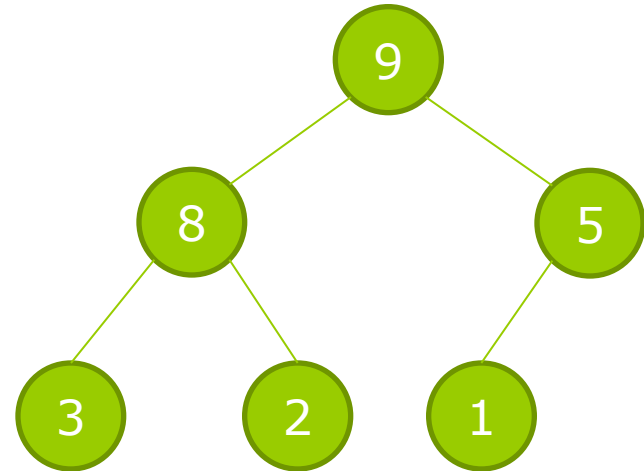
1 BUILD-MAX-HEAP(A, n)

2 **for $i = n$ downto 2**

3 exchange $A[1]$ with $A[i]$

4 $A.heap-size = A.heap-size - 1$

5 MAX-HEAPIFY($A, 1$)



The Heapsort algorithm: Example 1



HEAPSORT(A, n)

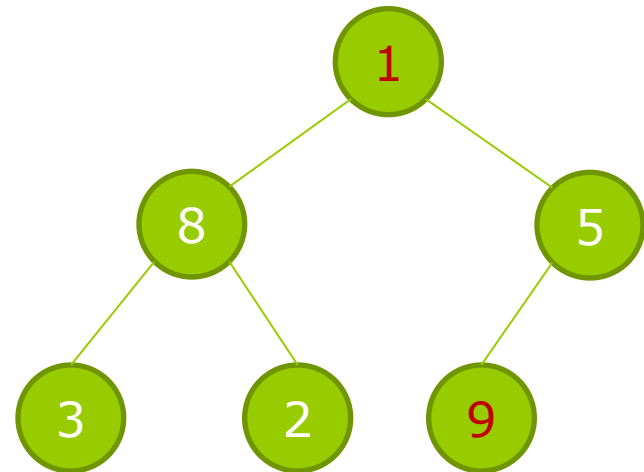
1 BUILD-MAX-HEAP(A, n)

2 **for** $i = n$ **downto** 2

3 exchange $A[1]$ with $A[i]$

4 $A.heap-size = A.heap-size - 1$

5 MAX-HEAPIFY($A, 1$)



The Heapsort algorithm: Example 1



HEAPSORT(A, n)

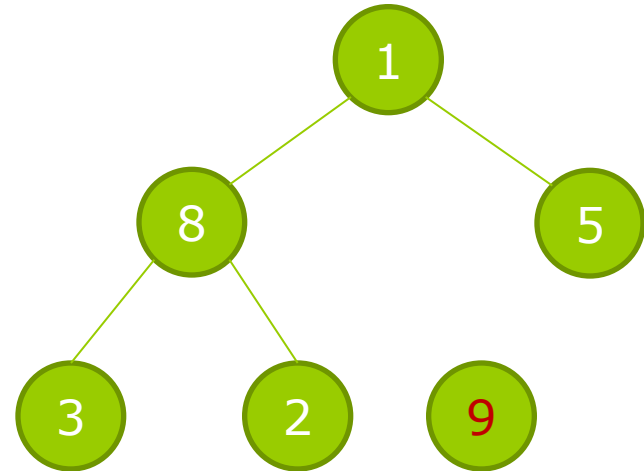
1 BUILD-MAX-HEAP(A, n)

2 **for** $i = n$ **downto** 2

3 exchange $A[1]$ with $A[i]$

4 $A.heap-size = A.heap-size - 1$

5 MAX-HEAPIFY($A, 1$)

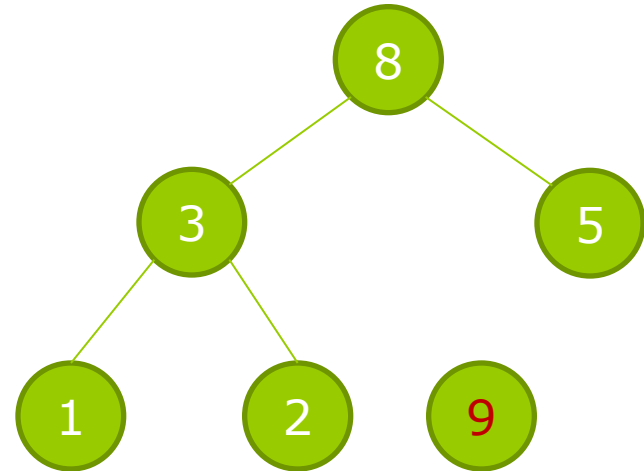


The Heapsort algorithm: Example 1

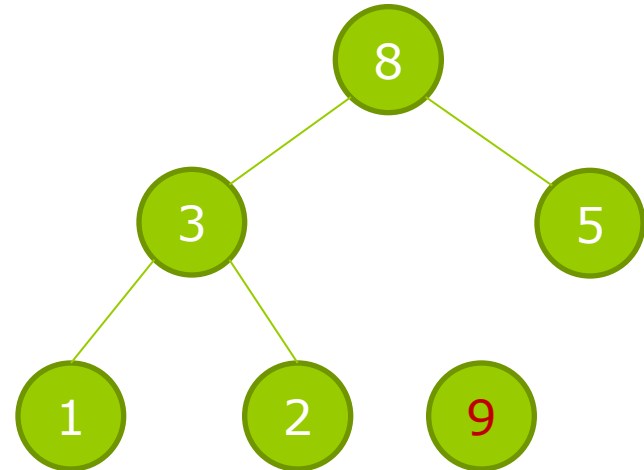


HEAPSORT(A, n)

```
1  BUILD-MAX-HEAP( $A, n$ )
2  for  $i = n$  downto 2
3      exchange  $A[1]$  with  $A[i]$ 
4       $A.heap-size = A.heap-size - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
```



The Heapsort algorithm: Example 1


$$\text{HEAPSORT}(A, n)$$
1 BUILD-MAX-HEAP(A, n)

```
2  for  $i = n$  downto 2
```

```

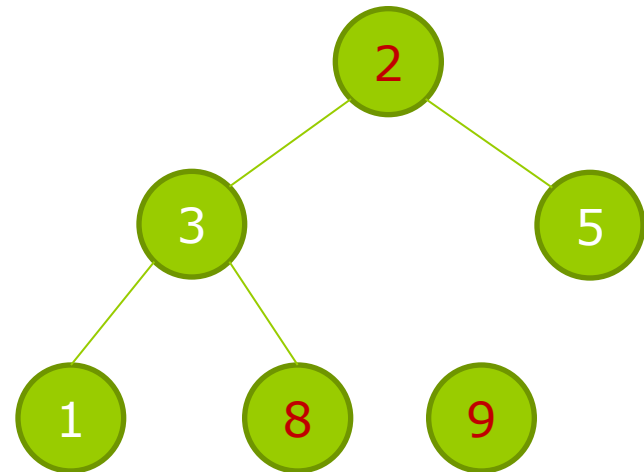
3         exchange A[1] with A[i]

```

$$4 \quad A.heap\text{-}size = A.heap\text{-}size - 1$$

5 **MAX-HEAPIFY**($A, 1$)

The Heapsort algorithm: Example 1


$$\text{HEAPSORT}(A, n)$$
1 BUILD-MAX-HEAP(A, n)

```
2  for  $i = n$  downto 2
```

```

3         exchange A[1] with A[i]

```

$$4 \quad \underline{A.heap-size = A.heap-size - 1}$$
5 MAX-HEAPIFY($A, 1$)

The Heapsort algorithm: Example 1



HEAPSORT(A, n)

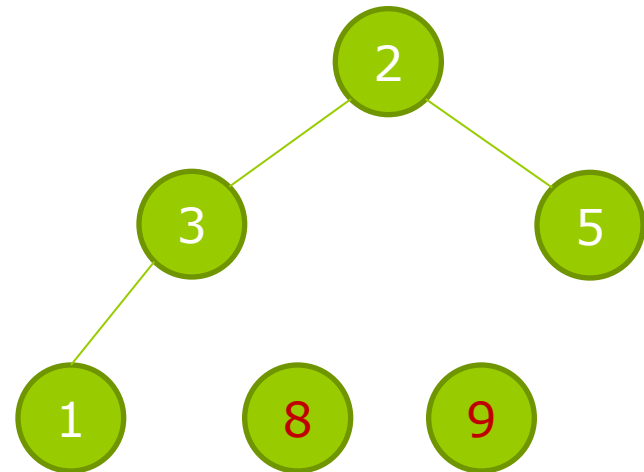
1 BUILD-MAX-HEAP(A, n)

2 **for** $i = n$ **downto** 2

3 exchange $A[1]$ with $A[i]$

4 $A.heap-size = A.heap-size - 1$

5 MAX-HEAPIFY($A, 1$)



The Heapsort algorithm: Example 1



HEAPSORT(A, n)

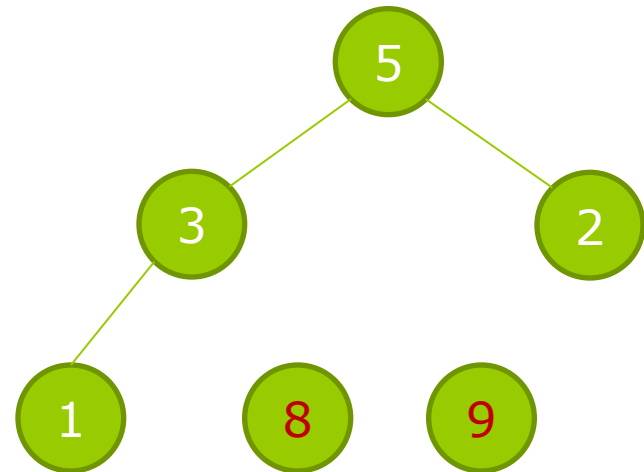
1 BUILD-MAX-HEAP(A, n)

2 **for** $i = n$ **downto** 2

3 exchange $A[1]$ with $A[i]$

4 $A.heap-size = A.heap-size - 1$

5 MAX-HEAPIFY($A, 1$)



The Heapsort algorithm: Example 1



HEAPSORT(A, n)

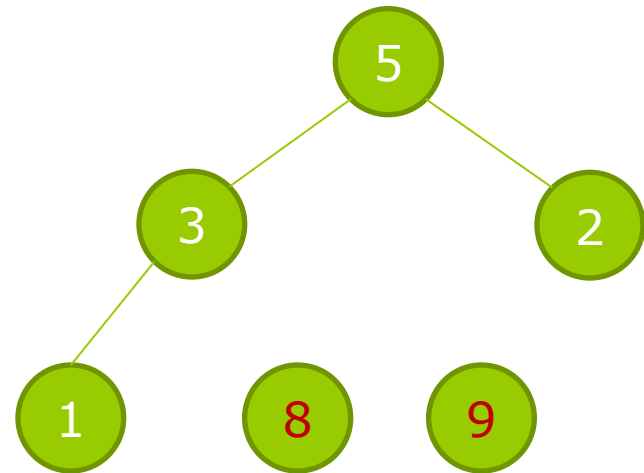
1 BUILD-MAX-HEAP(A, n)

2 **for $i = n$ downto 2**

3 exchange $A[1]$ with $A[i]$

4 $A.heap-size = A.heap-size - 1$

5 MAX-HEAPIFY($A, 1$)



The Heapsort algorithm: Example 1



HEAPSORT(A, n)

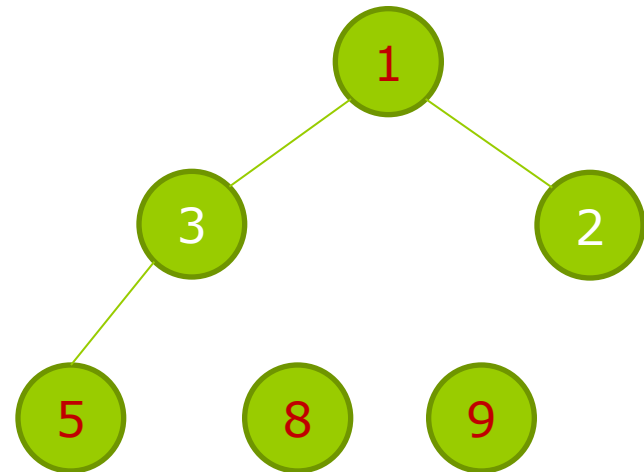
1 BUILD-MAX-HEAP(A, n)

2 **for** $i = n$ **downto** 2

3 exchange $A[1]$ with $A[i]$

4 $A.heap-size = A.heap-size - 1$

5 MAX-HEAPIFY($A, 1$)



The Heapsort algorithm: Example 1



HEAPSORT(A, n)

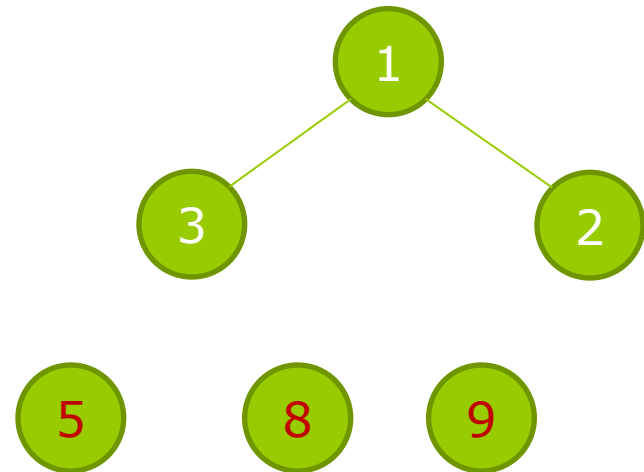
1 BUILD-MAX-HEAP(A, n)

2 **for** $i = n$ **downto** 2

3 exchange $A[1]$ with $A[i]$

4 $A.heap-size = A.heap-size - 1$

5 MAX-HEAPIFY($A, 1$)



The Heapsort algorithm: Example 1



HEAPSORT(A, n)

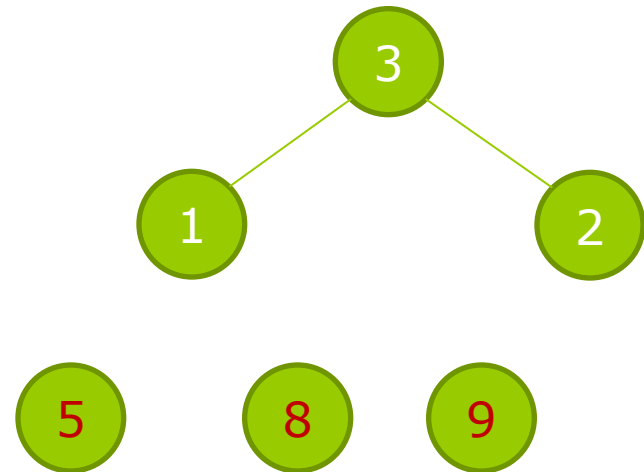
1 BUILD-MAX-HEAP(A, n)

2 **for** $i = n$ **downto** 2

3 exchange $A[1]$ with $A[i]$

4 $A.heap-size = A.heap-size - 1$

5 **MAX-HEAPIFY**($A, 1$)



The Heapsort algorithm: Example 1



HEAPSORT(A, n)

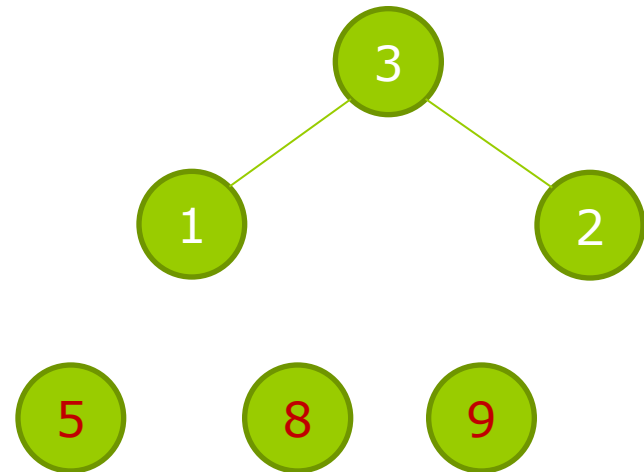
1 BUILD-MAX-HEAP(A, n)

2 **for $i = n$ downto 2**

3 exchange $A[1]$ with $A[i]$

4 $A.heap-size = A.heap-size - 1$

5 MAX-HEAPIFY($A, 1$)



The Heapsort algorithm: Example 1



HEAPSORT(A, n)

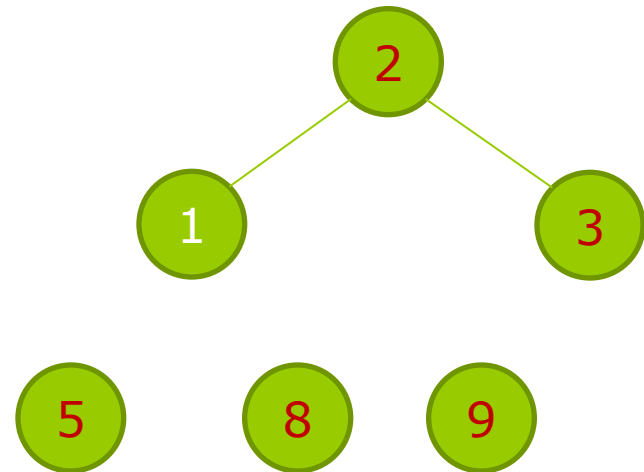
1 BUILD-MAX-HEAP(A, n)

2 **for** $i = n$ **downto** 2

3 exchange $A[1]$ with $A[i]$

4 $A.heap-size = A.heap-size - 1$

5 MAX-HEAPIFY($A, 1$)

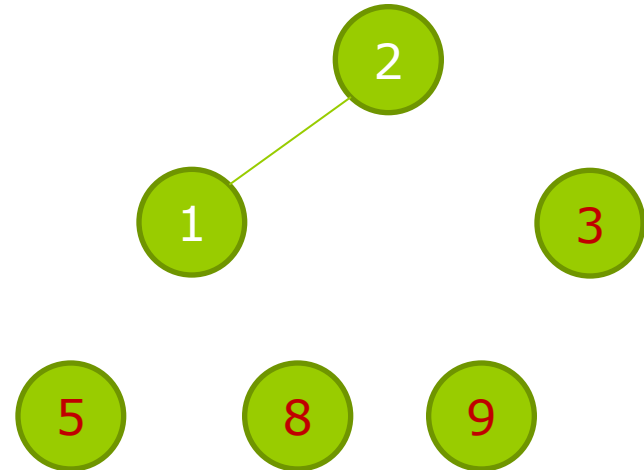


The Heapsort algorithm: Example 1



HEAPSORT(A, n)

```
1  BUILD-MAX-HEAP( $A, n$ )
2  for  $i = n$  downto 2
3      exchange  $A[1]$  with  $A[i]$ 
4       $A.heap-size = A.heap-size - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
```



The Heapsort algorithm: Example 1



HEAPSORT(A, n)

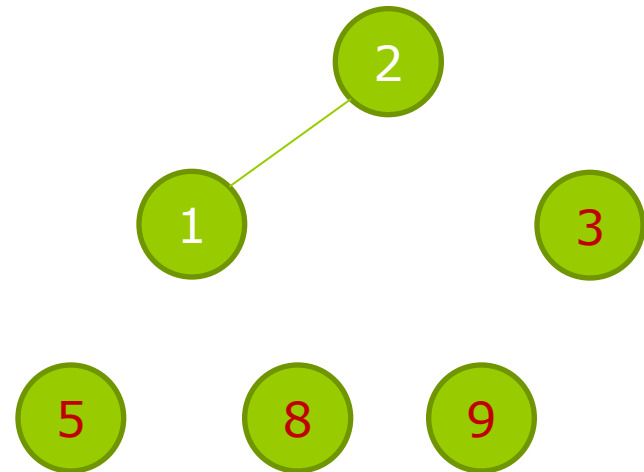
1 BUILD-MAX-HEAP(A, n)

2 **for** $i = n$ **downto** 2

3 exchange $A[1]$ with $A[i]$

4 $A.heap-size = A.heap-size - 1$

5 **MAX-HEAPIFY**($A, 1$)



The Heapsort algorithm: Example 1



HEAPSORT(A, n)

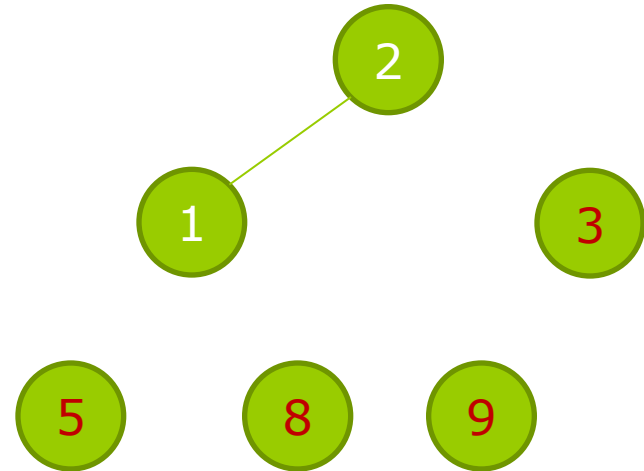
1 BUILD-MAX-HEAP(A, n)

2 **for $i = n$ downto 2**

3 exchange $A[1]$ with $A[i]$

4 $A.heap-size = A.heap-size - 1$

5 MAX-HEAPIFY($A, 1$)



The Heapsort algorithm: Example 1



HEAPSORT(A, n)

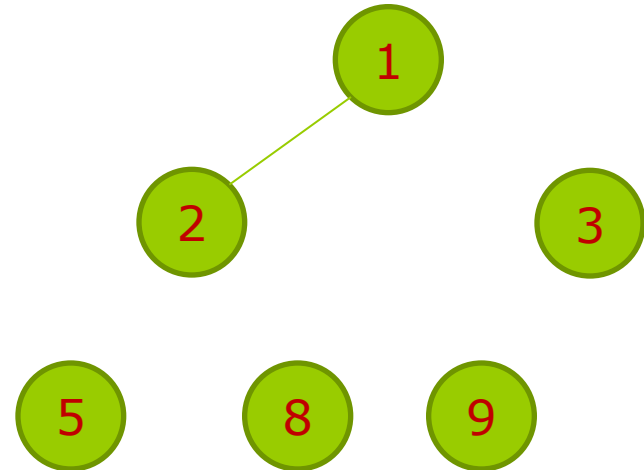
1 BUILD-MAX-HEAP(A, n)

2 **for** $i = n$ **downto** 2

3 exchange $A[1]$ with $A[i]$

4 $A.heap-size = A.heap-size - 1$

5 MAX-HEAPIFY($A, 1$)



The Heapsort algorithm: Example 1



HEAPSORT(A, n)

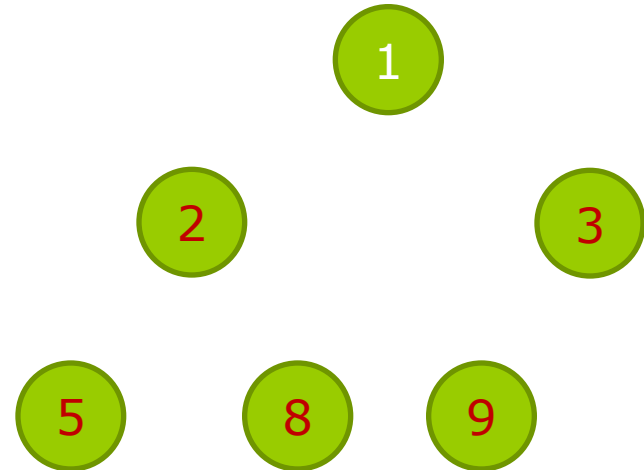
1 BUILD-MAX-HEAP(A, n)

2 **for** $i = n$ **downto** 2

3 exchange $A[1]$ with $A[i]$

4 $A.heap-size = A.heap-size - 1$

5 MAX-HEAPIFY($A, 1$)



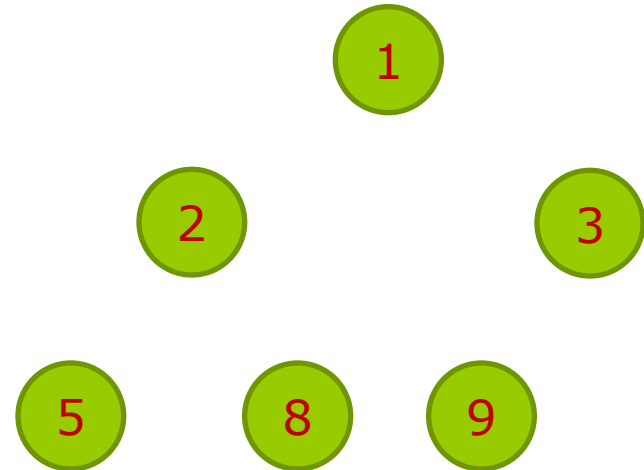
The Heapsort algorithm: Example 1

The array is fully sorted!

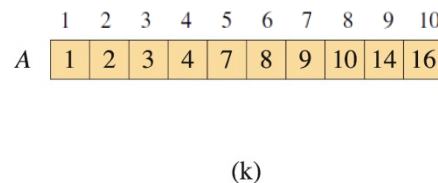
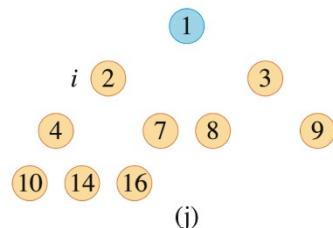
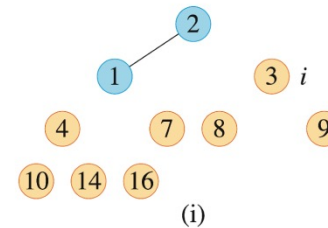
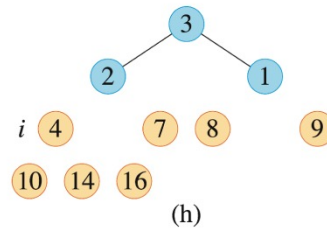
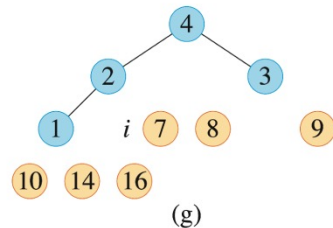
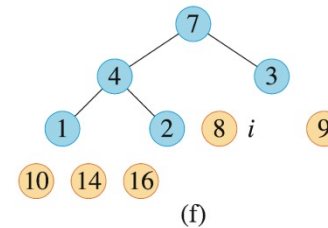
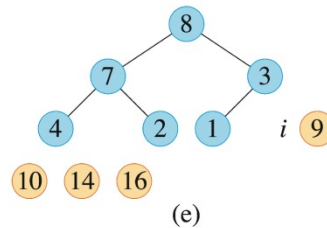
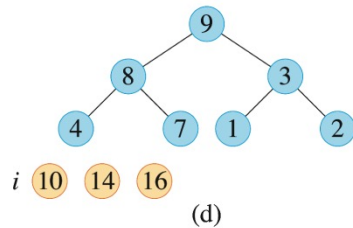
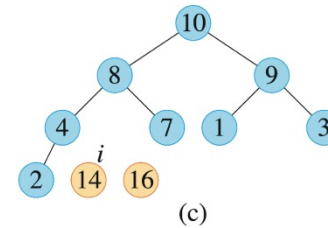
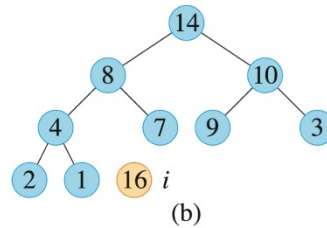
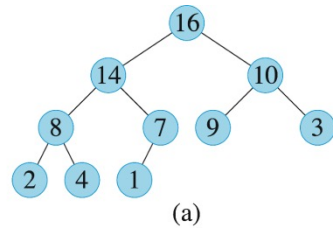


HEAPSORT(A, n)

```
1  BUILD-MAX-HEAP( $A, n$ )
2  for  $i = n$  downto 2
3      exchange  $A[1]$  with  $A[i]$ 
4       $A.heap-size = A.heap-size - 1$ 
5  MAX-HEAPIFY( $A, 1$ )
```



The Heapsort algorithm: Example 2



The Heapsort algorithm: Analysis

- ❑ The analysis of heapsort algorithm is as follows:
 - BUILD-MAX-HEAP: $O(n)$
 - **for** loop: $n - 1$ times
 - Exchange elements: $O(1)$
 - MAX-HEAPIFY: $O(\lg n)$
- ❑ Thus the HEAPSORT procedure takes $O(n \lg n)$ time.
- ❑ Though heapsort is a great algorithm, a well-implemented quicksort usually beats it in practice.

Priority queues

- A *priority queue* is a data structure for maintaining a set S of elements, each with an associated value called a *key*. A *max-priority queue* supports the following operations:
 - INSERT(S, x, k): inserts the element x with key k into the set S , which is equivalent to the operation $S = S \cup \{x\}$.
 - MAXIMUM(S): returns the elements of S with the largest key.
 - EXTRACT-MAX(S): removes and returns the element of S with the largest key.
 - INCREASE-KEY(S, x, k): increases the value of element x 's key to the new value k , which is assumed to be at least as large as x 's current key value.

- Example max-priority queue application: schedule jobs on shared computer. Scheduler adds new jobs to run by calling INSERT and runs the job with the highest priority among those pending by calling EXTRACT-MAX.

Priority queues

- ❑ Elements in the priority queue correspond to objects in the application that uses the priority queue. Each object contains a key. Need to be able to map between application objects and array indices in the heap. Two suggested ways:
 - Each heap element has a *handle* that allows access to an object in the application, and each object in the application has a handle (likely an array index) to access the heap element.
 - Store within the priority queue a mapping from application objects to array indices in the heap.

Advantage: application object do not need to use handles.

Disadvantage: need to establish and maintain the mapping.

One option would be to use a hash table, which is how Python dictionaries are implemented.

Priority queues: Finding the maximum element

- Getting the maximum element is easy: it is the root. First check that the heap is not empty.

MAX-HEAP-MAXIMUM(A)

```
1  if  $A.heap-size < 1$   
2      error “heap underflow”  
3  return  $A[1]$ 
```

- Time: $\Theta(1)$.

Priority queues:

Extracting the maximum element

- Given the array A :
 - Identify the maximum element by calling MAX-HEAP-MAXIMUM.
 - Make the last node in the tree the new root.
 - Remove the last node from the heap by decrementing *heap-size*.
 - Re-heapify the heap by calling MAX-HEAPIFY. Implicitly assume that MAX-HEAPIFY compares objects based on keys, and also that it updates the mapping between objects and array indices as necessary.
 - Return the copy of the maximum element.

MAX-HEAP-EXTRACT-MAX(A)

```
1   $max = \text{MAX-HEAP-MAXIMUM}(A)$ 
2   $A[1] = A[A.\text{heap-size}]$ 
3   $A.\text{heap-size} = A.\text{heap-size} - 1$ 
4  MAX-HEAPIFY( $A, 1$ )
5  return  $max$ 
```

- Analysis: Constant-time assignments plus time for MAX-HEAPIFY.
- Time: $O(\lg n)$.

Priority queues:

Increasing the value of an object's key

- Given set S , object x , and new key value k :
 - Make sure $k \geq x$'s current key.
 - Update x 's key value to k .
 - Find where in the array where x occurs.
 - Traverse the tree upward comparing the key to its parent's key and swapping keys if necessary, until the node's key is smaller than its parent's key or reach the root. Update the mapping between objects and array indices as necessary.

MAX-HEAP-INCREASE-KEY(A, x, k)

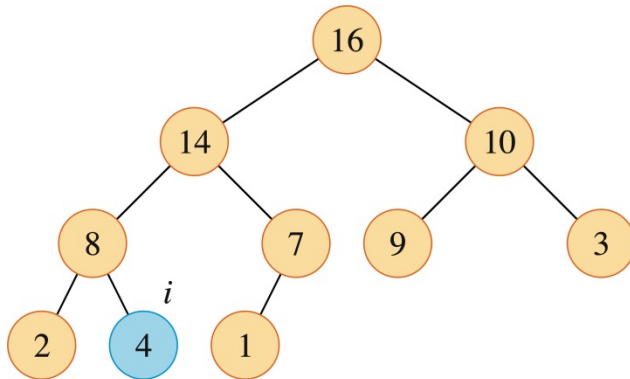
```
1  if  $k < x.key$ 
2      error "new key is smaller than current key"
3   $x.key = k$ 
4  find the index  $i$  in array  $A$  where object  $x$  occurs
5  while  $i > 1$  and  $A[PARENT(i)].key < A[i].key$ 
6      exchange  $A[i]$  with  $A[PARENT(i)]$ , updating the information that maps
          priority queue objects to array indices
7       $i = PARENT(i)$ 
```

- Analysis: Upward path from node i has length $O(\lg n)$ in an n -element heap.
- Time: $O(\lg n)$.

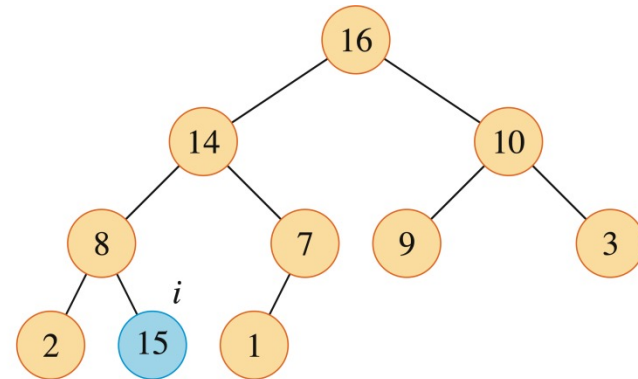
Priority queues:

Increasing the value of an object's key

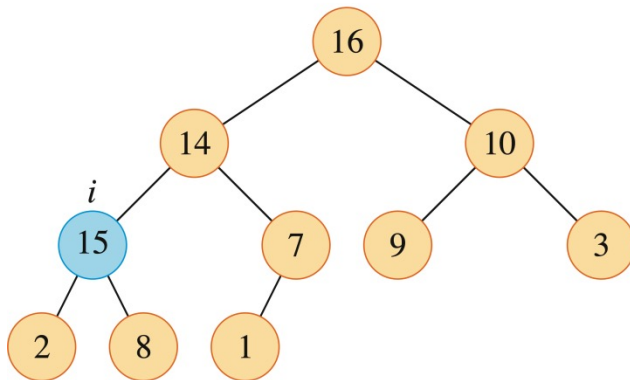
- *Example:* Increase key of node i to have value 15.



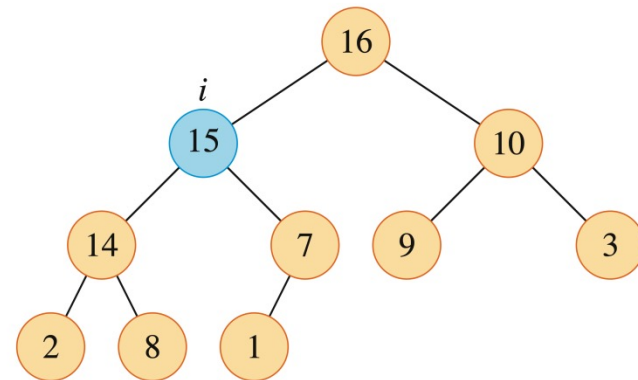
(a)



(b)



(c)



(d)

Priority queues:

Inserting into the heap

- Given an object x to insert into the heap:
 - Check that the heap has space for a new object.
 - Add a new node to the heap by incrementing *heap-size*.
 - Insert a new node in the last position in the heap, with key $-\infty$.
 - Save the value of x 's key in the variable k , and set x 's key to $-\infty$.
 - Make x be the last node in the heap, updating the mapping between objects and array indices as necessary.
 - Increase the $-\infty$ key to k using the HEAP-INCREASE-KEY procedure defined above.

MAX-HEAP-INSERT(A, x, n)

```
1  if  $A.heap-size == n$ 
2      error "heap overflow"
3   $A.heap-size = A.heap-size + 1$ 
4   $k = x.key$ 
5   $x.key = -\infty$ 
6   $A[A.heap-size] = x$ 
7  map  $x$  to index heap-size in the array
8  MAX-HEAP-INCREASE-KEY( $A, x, k$ )
```

- Analysis: Constant-time assignments plus time for HEAP-INCREASE-KEY.
- Time: $O(\lg n)$.

Reading

- ▣ Sections 6.1 ~ 6.5